UNIT 3 · TOPIC 3.4

Firewall Access Controls

Assignment, Answer Key

Teacher answer key, 22 points total.

Practice Questions

3.4.a · Firewall types (stateful vs. stateless)

1. A firewall examines each packet independently without remembering previous packets. Which type is this? (1 pt)

- ○ A. Stateful firewall
- **● B. Stateless (packet-filter) firewall**
- ○ C. Application-layer firewall
- ○ D. Intrusion detection system

Why: A stateless (packet-filter) firewall evaluates each packet in isolation using only header fields, with no memory of prior packets or connections.

2. Which data structure does a stateful firewall use to recognize legitimate return traffic? (1 pt)

- ○ A. Routing table
- ○ B. ARP cache
- ○ C. MAC address table
- **● D. Connection-tracking table**

Why: The connection-tracking table records active sessions (source/destination IP, ports, protocol, state). Return packets matching an established entry are automatically permitted.

3. Select ALL fields a stateless packet-filter firewall can use in its rules. (3 pts)

- **■ A. Source IP address**
- **■ B. Destination port number**
- □ C. TCP connection state (ESTABLISHED)
- **■ D. Protocol (TCP/UDP)**

Why: Stateless firewalls operate on header fields: source/destination IP, port numbers, and protocol. They cannot inspect TCP connection state because they track no state.

4. An attacker sends a TCP packet with the ACK flag set, spoofing a session that was never established. A stateful firewall will: (1 pt)

- ○ A. Forward it because ACK packets are always allowed
- **● B. Drop it because no matching session exists in the connection-tracking table**
- ○ C. Forward it because the destination port matches a permit rule
- ○ D. Drop it because ACK packets are never allowed inbound

Why: A stateful firewall checks whether the packet belongs to a tracked session. An ACK for a session not in the table is dropped as an unsolicited or spoofed packet.

5. Name the three-packet exchange that a stateful firewall uses to detect the start of a legitimate TCP connection. (1 pt)

Acceptable answers: three-way handshake; SYN SYN-ACK ACK; TCP handshake; syn synack ack

Why: The TCP three-way handshake (SYN, SYN-ACK, ACK) establishes a connection. Stateful firewalls track this sequence to know when a session is legitimately open.

6. Which scenario BEST justifies using a stateless firewall instead of a stateful one? (1 pt)

- ● **A. High-throughput edge router where simple IP/port rules and raw speed are the priority**
- ○ B. Protecting an internal database server from spoofed packets
- ○ C. Filtering encrypted HTTPS traffic for malware
- ○ D. Tracking which users are currently logged in to the network

Why: Stateless firewalls excel where raw throughput matters and simple permit/deny rules are sufficient, such as high-speed edge routers. Richer inspection requires stateful or application-layer firewalls.

3.4.b · Firewall rules and access control lists (ACLs)

7. A firewall ACL evaluates a packet against its rule list. Which rule's action is applied? (1 pt)

- ○ A. The last rule that matches
- ○ B. The rule with the most specific source address
- ○ C. All matching rules, combined
- ● **D. The first rule (top to bottom) that matches**

Why: ACLs use first-match-wins: evaluation stops at the first rule whose conditions match the packet, regardless of any later rules that might also match.

8. A packet reaches the bottom of an ACL without matching any explicit rule. What happens? (1 pt)

- ○ A. It is allowed by default
- ○ B. The firewall crashes
- ● **C. It is dropped by the implicit deny**
- ○ D. It is queued for manual review

Why: A well-formed ACL ends with an implicit (or explicit) deny-all rule. Unmatched traffic is dropped by default.

9. Select ALL fields a typical ACL rule matches on. (3 pts)

- ■ **A. Source IP/subnet**
- ■ **B. Destination port**
- ■ **C. Protocol (TCP/UDP)**
- □ D. The packet's payload text

Why: ACL rules match header fields: source, destination, port, and protocol. Inspecting payload content is beyond a basic ACL (that is application-layer inspection).

10. An admin places 'allow TCP any any port 22' as rule 1, then 'deny TCP 203.0.113.0 to 10.0.1.10 port 22' as rule 4. What happens to traffic from 203.0.113.0 to port 22? (1 pt)

- ○ A. It is denied, because rule 4 is more specific
- ○ B. The firewall evaluates both rules and denies since deny takes priority
- ● **C. It is allowed, because rule 1 matches first and rule 4 is shadowed**
- ○ D. It depends on which rule was written most recently

Why: Rule 1 is broader and matches first, so its allow action applies. Rule 4 is unreachable (shadowed) for that traffic, specificity does not override rule order.

11. Name the anomaly where a broad rule earlier in an ACL makes a more specific rule below it unreachable. (1 pt)

Acceptable answers: shadowing; rule shadowing; shadowed rule

Why: Rule shadowing: an earlier, broader-matching rule prevents a later, more specific rule from ever being evaluated.

12. What is the safest general pattern for ordering ACL rules? (1 pt)

- ○ A. Broadest allow rules first, specific denies last
- ○ B. Alphabetical order by source IP
- ● **C. Most specific rules (denies/exceptions) first, general allows next, implicit deny-all last**
- ○ D. Order does not matter if the rule set is small

Why: Placing specific exceptions and denies before general allow rules prevents shadowing and keeps the default-deny posture intact.

Coding Assignments (Python 3)

3.4.a · Firewall types (stateful vs. stateless)

3 pts · auto-graded on the site

Coding problem 3.4.a: Stateless vs. stateful packet classifier

Skill: Mitigate Risk · **Maps to:** 3.4.A · **Language:** Python 3 (pytest)

Problem

You will implement two firewall decision functions that together illustrate the core difference between stateless and stateful packet filtering.

A **packet** is a dict with these string keys:

- `"src"`: source IP string (e.g. `"203.0.113.10"`)
- `"dst"`: destination IP string
- `"sport"`: source port integer
- `"dport"`: destination port integer
- `"proto"`: protocol string, one of `"tcp"` / `"udp"` / `"icmp"`
- `"flags"`: TCP flag string: one of `"SYN"`, `"SYN-ACK"`, `"ACK"`, `"RST"`, `"FIN"`, `"DATA"` (non-TCP packets use `""`)

A **rule** is a dict with:

- `"action"`, `"allow"` or `"drop"`
- `"proto"`: protocol to match (`"tcp"`, `"udp"`, `"icmp"`, or `"any"`)
- `"dport"`: destination port to match (integer) or `0` meaning any port

Rules are evaluated top-to-bottom; the first match decides. If no rule matches, the implicit verdict is `"drop"`.

Function contract

Implement `solve(rules, packets, mode)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

- `rules`: list of rule dicts (ordered, first-match)
- `packets`: list of packet dicts (process in order)
- `mode`: string: `"stateless"` or `"stateful"`

Return a `list[str]` of verdicts (`"allow"` or `"drop"`), one per packet, in order.

Stateless mode

For each packet, walk the rule list. A rule matches when: 1. `rule["proto"]` is `"any"` OR equals `packet["proto"]` 2. `rule["dport"]` is `0` OR equals `packet["dport"]`

Return the matched rule's action, or `"drop"` if nothing matches.

Stateful mode

Same rule matching applies, but also maintain an in-memory connection-tracking table (a dict). The table key is `(src, dst, sport, dport, proto)` (tuple of the five fields above).

Additional stateful logic for TCP packets only:

- **SYN**: if the packet passes rules, add `(src, dst, sport, dport, "tcp")` to the table with state `"SYN_SENT"`. Verdict = `"allow"`.
- **SYN-ACK**: look up the *reverse* key `(dst, src, dport, sport, "tcp")` in the table (the original SYN entry). If found, update its state to `"ESTABLISHED"`. Verdict = `"allow"`. If the reverse key is not in the table, verdict = `"drop"`.
- **ACK / DATA**: if the packet's key OR its reverse is in the table with state `"ESTABLISHED"`, verdict = `"allow"`. Otherwise verdict = `"drop"` (no established session for this endpoint pair, a stateful firewall drops it even though a static rule might otherwise permit the port).
- **RST / FIN**: if the key or reverse is in the table, remove the entry. Verdict = `"allow"`. Otherwise `"drop"`.

Non-TCP packets (UDP, ICMP) are decided by rule matching only (no state tracking).

The table starts empty at the beginning of each `solve()` call.

Input

```
rules = [
    {"action": "allow", "proto": "tcp", "dport": 443},
    {"action": "allow", "proto": "tcp", "dport": 80},
    {"action": "drop",  "proto": "any", "dport": 0},
]
packets = [
    {"src": "203.0.113.10", "dst": "10.0.0.5", "sport": 55001, "dport": 443, "proto": "tcp", "flags": "SYN"},
    {"src": "10.0.0.5",     "dst": "203.0.113.10", "sport": 443, "dport": 55001, "proto": "tcp", "flags": "SYN-
ACK"},
    {"src": "198.51.100.7", "dst": "10.0.0.5", "sport": 44000, "dport": 443, "proto": "tcp", "flags": "ACK"},
]
```

Output

```
stateless: ["allow", "drop", "allow"]   # SYN-ACK (dport 55001) matches no rule
stateful:  ["allow", "allow", "drop"]   # third packet: ACK with no prior SYN
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).
- The connection table is local to each `solve()` call (not persisted between calls).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import List, Dict, Any


def _match_rules(rules: List[Dict], packet: Dict) -> str:
    for rule in rules:
        proto_ok = rule["proto"] == "any" or rule["proto"] == packet["proto"]
        dport_ok = rule["dport"] == 0 or rule["dport"] == packet["dport"]
        if proto_ok and dport_ok:
            return rule["action"]
    return "drop"


def solve(rules: List[Dict], packets: List[Dict], mode: str) -> List[str]:
    if mode == "stateless":
        return [_match_rules(rules, pkt) for pkt in packets]

    table: Dict[tuple, str] = {}
    verdicts: List[str] = []

    for pkt in packets:
        proto = pkt["proto"]
        flags = pkt.get("flags", "")
        key = (pkt["src"], pkt["dst"], pkt["sport"], pkt["dport"], proto)
        rev = (pkt["dst"], pkt["src"], pkt["dport"], pkt["sport"], proto)

        if proto != "tcp":
            verdicts.append(_match_rules(rules, pkt))
            continue

        if flags == "SYN":
            rule_verdict = _match_rules(rules, pkt)
            if rule_verdict == "allow":
                table[key] = "SYN_SENT"
                verdicts.append("allow")
            else:
                verdicts.append("drop")

        elif flags == "SYN-ACK":
            if rev in table:
                table[rev] = "ESTABLISHED"
                verdicts.append("allow")
            else:
                verdicts.append("drop")

        elif flags in ("ACK", "DATA"):
            if table.get(key) == "ESTABLISHED" or table.get(rev) == "ESTABLISHED":
                verdicts.append("allow")
            else:
                verdicts.append("drop")

        elif flags in ("RST", "FIN"):
            if key in table or rev in table:
                table.pop(key, None)
                table.pop(rev, None)
                verdicts.append("allow")
            else:
                verdicts.append("drop")

        else:
            verdicts.append(_match_rules(rules, pkt))

    return verdicts
```

3.4.b · Firewall rules and access control lists (ACLs)

3 pts · auto-graded on the site

Coding problem 3.4.b: ACL evaluator (first-match-wins)

Skill: Mitigate Risk · **Maps to:** 3.4.B, 3.4.C · **Language:** Python 3 (pytest)

Problem

Implement a firewall ACL evaluator. Given an ordered list of rules and a list of packets, decide the verdict ("allow" or "deny") for each packet using first-match-wins evaluation with an implicit deny-all at the end of the list.

A **rule** is a dict with these string/number keys:

- `"action"`, `"allow"` or `"deny"`
- `"src"`: source IP string, or `"any"`
- `"dst"`: destination IP string, or `"any"`
- `"port"`: destination port integer, or `0` meaning any port
- `"proto"`: protocol string: `"tcp"`, `"udp"`, or `"any"`

A **packet** is a dict with:

- `"src"`: source IP string
- `"dst"`: destination IP string
- `"port"`: destination port integer
- `"proto"`: protocol string (`"tcp"` or `"udp"`)

Function contract

Implement `solve(rules, packets)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

- `rules`: ordered list of rule dicts (evaluate top to bottom, first match wins)
- `packets`: list of packet dicts (evaluate independently, in order)

Return a `list[str]` of verdicts (`"allow"` or `"deny"`), one per packet, in the same order as `packets`.

Rule matching

A rule matches a packet when ALL of the following hold: 1. `rule["src"] == "any"` OR `rule["src"] == packet["src"]` 2. `rule["dst"] == "any"` OR `rule["dst"] == packet["dst"]` 3. `rule["port"] == 0` OR `rule["port"] == packet["port"]` 4. `rule["proto"] == "any"` OR `rule["proto"] == packet["proto"]`

Walk `rules` in order; return the **first** matching rule's `"action"`. If no rule in the list matches, the verdict is `"deny"` (the implicit deny-all, you do not need a literal rule for this; just return `"deny"` when nothing matches).

Examples

```
rules = [
    {"action": "allow", "src": "10.0.4.0", "dst": "10.0.1.5", "port": 3306, "proto": "tcp"},
    {"action": "deny",  "src": "any",      "dst": "10.0.1.5", "port": 3306, "proto": "tcp"},
]
packets = [
    {"src": "10.0.4.0", "dst": "10.0.1.5", "port": 3306, "proto": "tcp"},   # matches rule 1 -> allow
```

```
    {"src": "10.0.9.9", "dst": "10.0.1.5", "port": 3306, "proto": "tcp"},   # matches rule 2 -> deny
    {"src": "10.0.9.9", "dst": "10.0.1.5", "port": 22,   "proto": "tcp"},   # matches nothing -> deny
]
solve(rules, packets) == ["allow", "deny", "deny"]
```

Shadowing example

```
rules = [
    {"action": "allow", "src": "any", "dst": "any", "port": 22, "proto": "tcp"},   # too broad, placed first
    {"action": "deny",  "src": "203.0.113.0", "dst": "10.0.1.10", "port": 22, "proto": "tcp"},
]
packet = {"src": "203.0.113.0", "dst": "10.0.1.10", "port": 22, "proto": "tcp"}
# The broad rule 1 matches first, so the deny in rule 2 never fires:
solve(rules, [packet]) == ["allow"]
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import List, Dict


def _rule_matches(rule: Dict, packet: Dict) -> bool:
    if rule["src"] != "any" and rule["src"] != packet["src"]:
        return False
    if rule["dst"] != "any" and rule["dst"] != packet["dst"]:
        return False
    if rule["port"] != 0 and rule["port"] != packet["port"]:
        return False
    if rule["proto"] != "any" and rule["proto"] != packet["proto"]:
        return False
    return True


def solve(rules: List[Dict], packets: List[Dict]) -> List[str]:
    verdicts: List[str] = []
    for pkt in packets:
        verdict = "deny"
        for rule in rules:
            if _rule_matches(rule, pkt):
                verdict = rule["action"]
                break
        verdicts.append(verdict)
    return verdicts
```